

DDA5001 Machine Learning

Unsupervised Learning: Dimensionality Reduction

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Agenda

Dimensionality Reduction

Principal Component Analysis (PCA)

Apply PCA to Real Image Dataset

Dimensionality Reduction

- Observe samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, without labels

Dimensionality reduction: Find a closest point to $\mathbf{x}_i$ in a lower dimensional space, i.e.,

$$\mathbf{x}_i \rightarrow \boldsymbol{\theta}_i \in \mathbb{R}^k,$$

where $k \ll d$.

Dimensionality Reduction

- Observe samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, without labels

Dimensionality reduction: Find a closest point to $\mathbf{x}_i$ in a lower dimensional space, i.e.,

$$\mathbf{x}_i \rightarrow \boldsymbol{\theta}_i \in \mathbb{R}^k,$$

where $k \ll d$.

- Contrary to kernel methods in supervised learning.

Dimensionality Reduction

- Observe samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, **without labels**

Dimensionality reduction: Find a **closest** point to $\mathbf{x}_i$ in a **lower dimensional space**, i.e.,

$$\mathbf{x}_i \rightarrow \boldsymbol{\theta}_i \in \mathbb{R}^k,$$

where $k \ll d$.

- Contrary to kernel methods in supervised learning.

The motivation of dimensionality reduction:

- Reducing redundant information.
- Help algorithms to be more computationally efficient (in lower dimension).
- Preventing overfitting, especially when $n < d$ (data preprocessing for supervised learning).

Dimensionality Reduction

- Observe samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, **without labels**

Dimensionality reduction: Find a **closest** point to $\mathbf{x}_i$ in a **lower dimensional space**, i.e.,

$$\mathbf{x}_i \rightarrow \boldsymbol{\theta}_i \in \mathbb{R}^k,$$

where $k \ll d$.

- Contrary to kernel methods in supervised learning.

The motivation of dimensionality reduction:

- Reducing redundant information.
- Help algorithms to be more computationally efficient (in lower dimension).
- Preventing overfitting, especially when $n < d$ (data preprocessing for supervised learning).

Dimensionality reduction is an important **unsupervised learning** technique. The main methods for dimensionality reduction are **feature selection** and **feature extraction**. We will focus on the latter.

Agenda

Dimensionality Reduction

Principal Component Analysis (PCA)

Apply PCA to Real Image Dataset

Principal Component Analysis

Principal component analysis (PCA): Find a low-dimensional approximation of high-dimensional data by minimizing the squared norms of distances.

Principal Component Analysis

Principal component analysis (PCA): Find a **low-dimensional approximation** of high-dimensional data by **minimizing the squared norms of distances**.

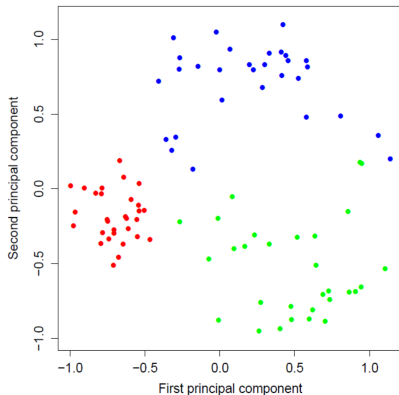
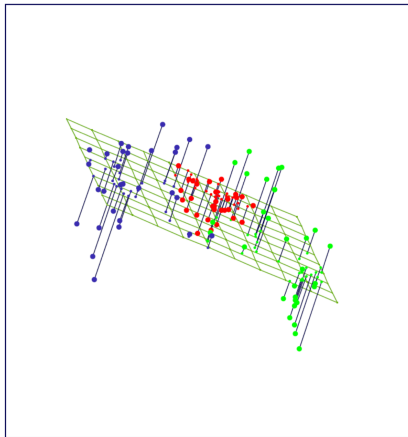
PCA modeling of data:

$$\mathbf{x}_i \approx \mathbf{A}\boldsymbol{\theta}_i + \boldsymbol{\mu}$$

- ▶ $\mathbf{x}_i \in \mathbb{R}^d$ is the original sample.
- ▶ $\mathbf{A} \in \mathbb{R}^{d \times k}$ with orthogonal columns, i.e., satisfying $\mathbf{A}^\top \mathbf{A} = \mathbf{I}_k$. Matrix $\mathbf{A}$ is often called **basis**.
- ▶ $\boldsymbol{\theta}_i \in \mathbb{R}^k$ is the **principle component**, i.e., **the lower-dimensional representation of $\mathbf{x}$** .
- ▶ $\boldsymbol{\mu}$ is the mean of the samples.

Interpretation: $\mathbf{x}_i$ (after removing the mean $\boldsymbol{\mu}$) can be approximated by the point $\mathbf{A}\boldsymbol{\theta}_i$, which lies in a k -dimensional subspace of $\mathbb{R}^d$.

Principal Component Analysis - Illustration



The PCA Learning Problem

PCA boils down to

$$\underset{\boldsymbol{\mu}, \{\boldsymbol{\theta}_i\}, \mathbf{A}^\top \mathbf{A} = \mathbf{I}_k}{\text{minimize}} \quad \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\boldsymbol{\theta}_i - \boldsymbol{\mu}\|_2^2$$

- ▶ Only samples $\{\mathbf{x}_i\}$ are known. Others are unknowns.
- ▶ It is a **nonconvex optimization** problem.
- ▶ The hard part is to solve for $\mathbf{A}$.
- ▶ Given $\mathbf{A}$, finding $\boldsymbol{\mu}$ and $\{\boldsymbol{\theta}_i\}$ is easy.

Solve for θ_i

Given A, μ

$$\underset{\{\theta_i\}}{\text{minimize}} \sum_{i=1}^n \|x_i - A\theta_i - \mu\|_2^2.$$

This reduces to n independent Least Squares problems with "data matrix" A and "labels" $x_i - \mu$.

Solution:

$$\theta_i = \underbrace{(A^\top A)^{-1}}_{A^\top A = I_k} A^\top (x_i - \mu) = A^\top (x_i - \mu)$$

Solve for μ

Suppose given A , setting $\theta_i = A^\top(x_i - \mu)$, we have

$$\underset{\mu}{\text{minimize}} \sum_{i=1}^n \|x_i - AA^\top(x_i - \mu) - \mu\|_2^2.$$

It is equivalent to

$$\underset{\mu}{\text{minimize}} \sum_{i=1}^n \|(\mathbf{I} - AA^\top)(x_i - \mu)\|_2^2.$$

It is further equivalent to

$$\underset{\mu}{\text{minimize}} \sum_{i=1}^n (x_i - \mu)^\top (\mathbf{I} - AA^\top)^\top (\mathbf{I} - AA^\top) (x_i - \mu).$$

Let $B = (\mathbf{I} - AA^\top)^\top (\mathbf{I} - AA^\top) = \mathbf{I} - AA^\top$.

Solve for μ

Take the gradient with respect to μ gives

$$\nabla_{\mu} = 2 \sum_{i=1}^n B(x_i - \mu).$$

Set the gradient to zero yields **one** solution

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i.$$

That is, the optimal μ is simply the mean of the samples.

Solve for A

It remains to solve

$$\underset{A^T A = I}{\text{minimize}} \sum_{i=1}^n \|x_i - AA^T(x_i - \mu) - \mu\|_2^2$$

- We can assume $\mu = 0$ without loss of generality, as we can set $x_i = x_i - \mu$ (removing the mean from the data).

Solve for $\mathbf{A}$

It remains to solve

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{minimize}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top(\mathbf{x}_i - \boldsymbol{\mu}) - \boldsymbol{\mu}\|_2^2$$

- We can assume $\boldsymbol{\mu} = 0$ without loss of generality, as we can set $\mathbf{x}_i = \mathbf{x}_i - \boldsymbol{\mu}$ (removing the mean from the data).

The problem reduces to

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{minimize}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top \mathbf{x}_i\|_2^2$$

This is one form of PCA.

- **Interpretation:** $\mathbf{A}\mathbf{A}^\top \mathbf{x}_i$ is the **projection** of $\mathbf{x}_i$ onto the k -dimensional subspace spanned by the columns of $\mathbf{A}$. This optimization problem corresponds to finding the k -dimensional subspace (i.e., finding $\mathbf{A}$) such that the **projected vector** $\mathbf{A}\mathbf{A}^\top \mathbf{x}_i$ is as close to $\mathbf{x}_i$ as possible.

Derivation of the Second Equivalent PCA Form

Given the PCA formulation

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{minimize}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A} \mathbf{A}^\top \mathbf{x}_i\|_2^2.$$

Derivation of the Second Equivalent PCA Form

Given the PCA formulation

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{minimize}} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top \mathbf{x}_i\|_2^2.$$

Expanding the objective function yields

$$\begin{aligned} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top \mathbf{x}_i\|_2^2 &= \sum_{i=1}^n (\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top \mathbf{x}_i)^\top (\mathbf{x}_i - \mathbf{A}\mathbf{A}^\top \mathbf{x}_i) \\ &= \sum_{i=1}^n \left(\mathbf{x}_i^\top \mathbf{x}_i - 2\mathbf{x}_i^\top \mathbf{A}\mathbf{A}^\top \mathbf{x}_i + \mathbf{x}_i^\top \mathbf{A}\mathbf{A}^\top \mathbf{A}\mathbf{A}^\top \mathbf{x}_i \right) \\ &= \sum_{i=1}^n \mathbf{x}_i^\top \mathbf{x}_i - \mathbf{x}_i^\top \mathbf{A}\mathbf{A}^\top \mathbf{x}_i. \end{aligned}$$

We reduce to the **second form** of PCA

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \sum_{i=1}^n \mathbf{x}_i^\top \mathbf{A}\mathbf{A}^\top \mathbf{x}_i = \sum_{i=1}^n \|\mathbf{A}^\top \mathbf{x}_i\|_2^2.$$

Derivation of the Third Equivalent PCA Form

The second form is further equivalent to

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \quad \sum_{i=1}^n \|\mathbf{A}^\top \mathbf{x}_i\|_2^2$$

$$\begin{array}{c} \Leftrightarrow \\ \underbrace{\|\mathbf{a}\|_2^2 = \text{trace}(\mathbf{a}\mathbf{a}^\top)} \end{array} \quad \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \quad \sum_{i=1}^n \text{trace}(\mathbf{A}^\top \mathbf{x}_i \mathbf{x}_i^\top \mathbf{A})$$

$$\begin{array}{c} \Leftrightarrow \\ \underbrace{\text{trace}(\mathbf{A}_1) + \text{trace}(\mathbf{A}_2) = \text{trace}(\mathbf{A}_1 + \mathbf{A}_2)} \end{array} \quad \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \quad \text{trace} \left(\mathbf{A}^\top \left(\sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top \right) \mathbf{A} \right).$$

Derivation of the Third Equivalent PCA Form

The second form is further equivalent to

$$\begin{aligned} & \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \quad \sum_{i=1}^n \|\mathbf{A}^\top \mathbf{x}_i\|_2^2 \\ & \quad \Leftrightarrow \quad \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \quad \sum_{i=1}^n \text{trace}(\mathbf{A}^\top \mathbf{x}_i \mathbf{x}_i^\top \mathbf{A}) \\ & \quad \Leftrightarrow \quad \underset{\text{trace}(\mathbf{A}_1) + \text{trace}(\mathbf{A}_2) = \text{trace}(\mathbf{A}_1 + \mathbf{A}_2)}{\text{maximize}} \quad \text{trace} \left(\mathbf{A}^\top \left(\sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top \right) \mathbf{A} \right). \end{aligned}$$

Let

$$\mathbf{S} = \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top = \mathbf{X} \mathbf{X}^\top.$$

be the **empirical covariance matrix**. We have the following **third form** of PCA

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \quad \text{trace}(\mathbf{A}^\top \mathbf{S} \mathbf{A}).$$

We use the third form to derive the solution for $\mathbf{A}$. We need to consult a matrix computation tool called **eigen decomposition**.

Eigenvalue Analysis

Eigenvalue problem: Given matrix S , find a vector u and a scalar λ such that

$$Su = \lambda u$$

- ▶ λ characterizes the behavior of S in u .
- ▶ u is called the **eigenvector**, while λ is called the **eigenvalue**.

Eigen decomposition for real PSD matrix

Suppose matrix $S \in \mathbb{R}^{d \times d}$ is **real** and **positive semidefinite (PSD)**, it always admits an eigen decomposition:

$$S = U \Lambda U^\top$$

where $U \in \mathbb{R}^{d \times d}$ is an orthogonal matrix satisfying $U^\top U = U U^\top = I$ containing eigenvectors as its columns and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$ with $\lambda_1 \geq \dots \geq \lambda_d \geq 0$ is a diagonal matrix containing the corresponding eigenvalues.

Return to The Solution of PCA

Consider the third form of PCA

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\mathbf{A}^\top \mathbf{S} \mathbf{A} \right),$$

where $\mathbf{S} = \mathbf{X} \mathbf{X}^\top$.

- ▶ Note that $\mathbf{S}$ is constructed from the data matrix $\mathbf{X}$, it is computable.
- ▶ Further, $\mathbf{S}$ is real and **must be PSD** (why?).

Thus we apply eigen decomposition to $\mathbf{S}$ to obtain

$$\mathbf{S} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top.$$

The PCA problem becomes

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\mathbf{A}^\top \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top \mathbf{A} \right).$$

Return to The Solution of PCA

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\mathbf{A}^\top \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top \mathbf{A} \right)$$

Let $\mathbf{\Phi} = \mathbf{A}^\top \mathbf{U} \in \mathbb{R}^{k \times d}$, it is a semi-orthogonal matrix since $\mathbf{\Phi} \mathbf{\Phi}^\top = \mathbf{I}$.

We can rewrite the optimization problem as

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\mathbf{\Phi} \mathbf{\Lambda} \mathbf{\Phi}^\top \right) = \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\sum_{i=1}^d \lambda_i \phi_i \phi_i^\top \right)$$

ϕ_i denotes the i^{th} column of $\mathbf{\Phi}$. Put the trace inside, we have

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \sum_{i=1}^d \lambda_i \phi_i^\top \phi_i$$

Return to The Solution of PCA

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\mathbf{A}^\top \mathbf{U} \mathbf{\Lambda} \mathbf{U}^\top \mathbf{A} \right)$$

Let $\mathbf{\Phi} = \mathbf{A}^\top \mathbf{U} \in \mathbb{R}^{k \times d}$, it is a semi-orthogonal matrix since $\mathbf{\Phi} \mathbf{\Phi}^\top = \mathbf{I}$. We can rewrite the optimization problem as

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\mathbf{\Phi} \mathbf{\Lambda} \mathbf{\Phi}^\top \right) = \underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \text{ trace} \left(\sum_{i=1}^d \lambda_i \phi_i \phi_i^\top \right)$$

ϕ_i denotes the i^{th} column of $\mathbf{\Phi}$. Put the trace inside, we have

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}}{\text{maximize}} \sum_{i=1}^d \lambda_i \phi_i^\top \phi_i$$

Fact: This optimization problem has upper bound $\leq \sum_{i=1}^k \lambda_i$. Hence, it attains its maximum when $\phi_i^\top \phi_i = 1, i = 1, \dots, k$ and $\phi_i^\top \phi_i = 0, i = k + 1, \dots, d$ (details omitted). This is achieved by

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k]$$

in which $\mathbf{u}_1, \dots, \mathbf{u}_k$ are the first k eigenvectors of $\mathbf{S}$.

Process of Computing The PCA

Given samples $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, **without labels**.

- ▶ Remove the mean

$$\mathbf{x}_i = \mathbf{x}_i - \boldsymbol{\mu},$$

where $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$.

- ▶ Form the empirical covariance matrix from data

$$\mathbf{S} = \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top = \mathbf{X} \mathbf{X}^\top.$$

- ▶ Compute the eigen decomposition of $\mathbf{S}$

$$\mathbf{S} = \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^\top$$

and the PCA solution is given by

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k].$$

- ▶ Compute the principle component (i.e., **the lower-dimensional projection**) and reconstructed data

$$\boldsymbol{\theta}_i = \mathbf{A}^\top \mathbf{x}_i, \quad \mathbf{A} \boldsymbol{\theta}_i = \mathbf{A} \mathbf{A}^\top \mathbf{x}_i.$$

Connection to Singular Value Decomposition

Singular value decomposition (SVD)

Given **any** real matrix $\mathbf{X} \in \mathbb{R}^{d \times n}$, there exists a 3-tuple $(\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}) \in \mathbb{R}^{d \times d} \times \mathbb{R}^{d \times n} \times \mathbb{R}^{n \times n}$ such that

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top,$$

where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal and $\mathbf{\Sigma}$ takes the form

$$\Sigma(i, j) = \begin{cases} \sigma_i, & i = j \\ 0, & i \neq j \end{cases}$$

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0, \quad p = \min(d, n).$$

- ▶ σ_i are called **singular value**.
- ▶ $\mathbf{u}_i$ and $\mathbf{v}_i$ are called **left and right singular vectors**.

PCA via SVD

Recall $\mathbf{S} = \mathbf{X}\mathbf{X}^\top$, the eigen decomposition is $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$, and the solution to the PCA problem is

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k]$$

PCA via SVD

Recall $S = \mathbf{X}\mathbf{X}^\top$, the eigen decomposition is $S = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$, and the solution to the PCA problem is

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k]$$

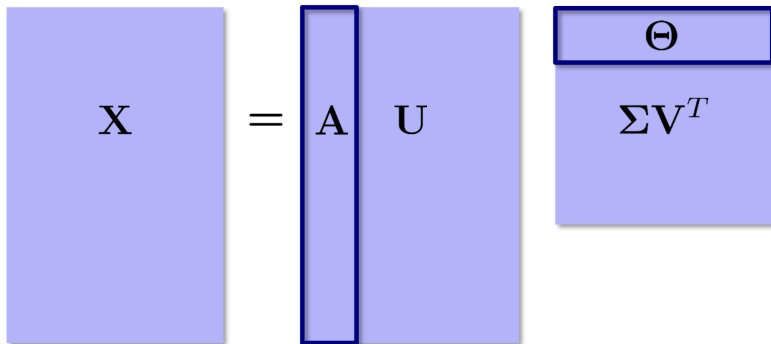
We can instead compute the SVD of matrix $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ and we also have (use $\mathbf{u}$ to denote the columns of $\mathbf{U}$)

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k].$$

- ▶ Computing PCA via SVD can be more favorable, as it can be more **numerically reliable** than eigen decomposition.
- ▶ It might also **save computation time** as we do not need to compute $S = \mathbf{X}\mathbf{X}^\top$, which can be expensive when $\mathbf{X}$ is a large matrix.

PCA via SVD: Illustration

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$



PCA From the Matrix Factorization Perspective

Recall the (first form) PCA (we assume without loss of generality that $\mu = 0$)

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \{\boldsymbol{\theta}_i\}}{\text{minimize}} \quad \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{A}\boldsymbol{\theta}_i\|_2^2.$$

It can be written in a matrix form:

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \boldsymbol{\Theta}}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\boldsymbol{\Theta}\|_F^2$$

where $\mathbf{A} \in \mathbb{R}^{d \times k}$ and $\boldsymbol{\Theta} \in \mathbb{R}^{k \times n}$, and $\|\cdot\|_F$ is the **Frobenius norm**.

- ▶ The above problem is also called **low-rank matrix factorization**.
- ▶ **Interpretation**: Factorize $\mathbf{X}$ into two factors' multiplication, where the latter is a **low-rank matrix**.

PCA From the Matrix Factorization Perspective

Low-rank matrix factorization

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \mathbf{\Theta}}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\mathbf{\Theta}\|_F^2$$

- Calculate the SVD of $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$.

Solution from PCA

- One optimal solution to the above (low-rank) matrix factorization problem is given by

$$\mathbf{A} = [\mathbf{u}_1, \dots, \mathbf{u}_k], \quad \mathbf{\Theta} = [\sigma_1 \mathbf{v}_1, \dots, \sigma_k \mathbf{v}_k]^\top$$

- It has infinitely many equivalent optimal solutions.

It is a **closed-form solution** to a nonconvex optimization problem.

More General Matrix Factorization

We can also remove the orthogonal constraint on $\mathbf{A}$ to allow more flexibility

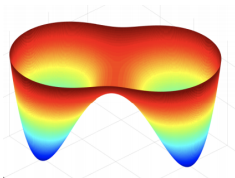
$$\underset{\mathbf{A} \in \mathbb{R}^{d \times k}, \mathbf{\Theta} \in \mathbb{R}^{k \times n}}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\mathbf{\Theta}\|_F^2.$$

One optimal solution to the above (low-rank) matrix factorization problem is given by

$$\mathbf{A} = [\sqrt{\sigma_1} \mathbf{u}_1, \dots, \sqrt{\sigma_k} \mathbf{u}_k], \quad \mathbf{\Theta} = [\sqrt{\sigma_1} \mathbf{v}_1, \dots, \sqrt{\sigma_k} \mathbf{v}_k]^\top,$$

and it has infinitely many equivalent optimal solution.

- ▶ A nonconvex optimization problem



- ▶ Closed-form solution exists.
- ▶ Recently, we have more advanced results (will introduce later).

LoRA: Low-rank Adaptation

In the **post-training** stage of large models (like large language models), we often need to learn an incremental update to the learned model to incorporate new knowledge. That is

$$\underset{\Delta\Theta \in \mathbb{R}^{m \times n}}{\text{minimize}} \quad \mathcal{L}(\hat{\Theta} + \Delta\Theta)$$

Low-rank Adaptation (LoRA) uses the simple idea of PCA. It does not aim to learn the full $\Delta\Theta$, it instead learns a **PCA decomposition** of $\Delta\Theta$.

$$\underset{A \in \mathbb{R}^{m \times r}, B \in \mathbb{R}^{r \times n}}{\text{minimize}} \quad \mathcal{L}(\hat{\Theta} + AB)$$

where $r \ll \min\{m, n\}$.

- ▶ LoRA approach can **save computation memory**.
- ▶ It is now widely utilized in large language models.
- ▶ We will use LoRA in our final project.

Agenda

Dimensionality Reduction

Principal Component Analysis (PCA)

Apply PCA to Real Image Dataset

The ORL Database of Faces

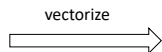


- ▶ 40 persons.
- ▶ Each has 10 distinct face images.
- ▶ Each image is of size 92×112 .
- ▶ The images were taken at different times, varying the lighting, facial expressions (open / closed eyes, smiling / not smiling) and facial details (glasses / no glasses).

Form the Data Matrix



92x112



vectorize

$\mathbf{x}_i$

$$\mathbf{x}_i = \begin{bmatrix} \mathbf{x}_i[1] \\ \vdots \\ \mathbf{x}_i[92 \times 112] \end{bmatrix} \in \mathbb{R}^{10304}$$

- Do the same thing for all the face images, we get

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n] \in \mathbb{R}^{d \times n},$$

where $d = 10304$, $n = 10 \times 40 = 400$.

Apply PCA to $\mathbf{X}$ with $k = 40$

Perform PCA, i.e., solving

$$\underset{\mathbf{A}^\top \mathbf{A} = \mathbf{I}, \mathbf{\Theta}}{\text{minimize}} \quad \|\mathbf{X} - \mathbf{A}\mathbf{\Theta}\|_F^2$$

We get the **extracted features** (i.e., $\mathbf{A} \in \mathbb{R}^{d \times k}$)



where we resize each column of $\mathbf{A}$ (of dimension $d = 10304$) to a 92×112 image and show it.

Application

$$\mathbf{x}_i = \mathbf{A}\boldsymbol{\theta}_i$$


- ▶ Each face image $\mathbf{x}_i$ can be interpreted as a linear combination of the columns in $\mathbf{A}$.
- ▶ The importance of each feature is implied in $\boldsymbol{\theta}_i$.
- ▶ $\mathbf{x}_i \in \mathbb{R}^d$ ($d = 10304$ here) has been nicely represented by a k -dimensional vector $\boldsymbol{\theta}_i$ ($k = 40$ here) — Dimensionality reduction.

↪ Next lecture: Unsupervised learning: Clustering and k-means.